Software Engineering Lab — Lab 5

Static Code Analysis Report & Evaluation

Course:	UE24CS341A — Software Engineering	**Date:**	October 30, 2026
Student Name:	Rohil Thimmanahalli Girish	**SRN:**	PES1UG24CS384
Department:	Computer Science & Engineering, PES University	**Assigned Scenario:**	Scenario 20: Scholarship Disbursement
Target Script:	scholarship_disbursement.py	**Git Repository:**	rohilgirish/SE-Labs-PES1UG24CS384 (Lab-5)

1. Introduction to Static Code Analysis & Core Tools

Static Code Analysis is a fundamental software engineering technique that analyzes source code without executing the underlying program. By traversing token streams and parsing code into an Abstract Syntax Tree (AST), static analyzers evaluate syntactic grammar, architectural conventions, security vulnerabilities, and code complexity. In modern DevOps and CI/CD pipelines, static analysis enforces quality gates early in the Software Development Life Cycle (SDLC) ('Shift Left'), detecting latent bugs before runtime testing or production deployment.

The Three Industry-Standard Tools Used in this Lab:

Tool	Primary Role & Scope	Analysis Methodology	Focus Areas & Metrics
Pylint	Strict Code Reviewer (Code Quality & Design)	Full AST traversal, type inference, semantic modeling	Identifies logical bugs, mutable default arguments, unused variables, design anti-patterns, and scores code from 0.00 to 10.00.
Bandit	Security Guard (Vulnerability Assessment)	Security AST node scanning, CWE signature matching	Detects security weaknesses such as SQL injection (CWE-89), command injection via shell execution (CWE-78), and silent exception swallowing (CWE-703).
Flake8	Style & Grammar Checker (PEP 8 Compliance)	Wraps PyFlakes (syntax), pycodestyle (style), and McCabe (complexity)	Enforces uniform Python styling, line length limits (≤79 characters), whitespace formatting, and proper comparison operators (e.g. is None).

Workflow Execution Summary for Lab 5:

The lab followed a rigorous four-phase workflow: **(1) Inspection & Baseline Execution:** Ran initial scans with Pylint, Bandit, and Flake8 and generated baseline reports. **(2) Issue Documentation:** Logged findings into a structured issue table prioritizing high/medium severity. **(3) Iterative Remediation via 3 Git Commits:** Committed fixes tool-by-tool (Bandit security fixes → Flake8 style fixes → Pylint quality refactoring). **(4) Verification & Reporting:** Re-ran static analysis confirming 0 security vulnerabilities, 0 style errors, and a perfect 10.00/10 score.

2. Issues Identified Before Fixing (Initial Scans)

The baseline script scholarship_disbursement.py was scanned using Bandit, Flake8, and Pylint prior to applying fixes. Below are the terminal captures demonstrating the findings reported by each tool.

Figure 1: Bandit Initial Scan (4 security issues: 1 Medium SQL injection CWE-89, 3 Low)

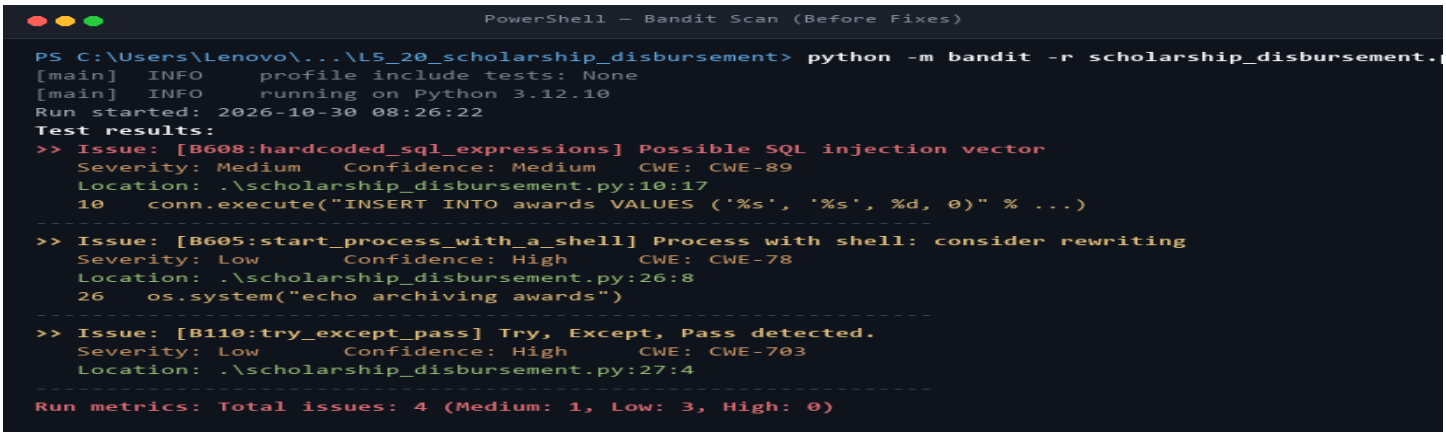


Figure 2: Flake8 Initial Scan (6 PEP 8 style violations detected across 32 lines)

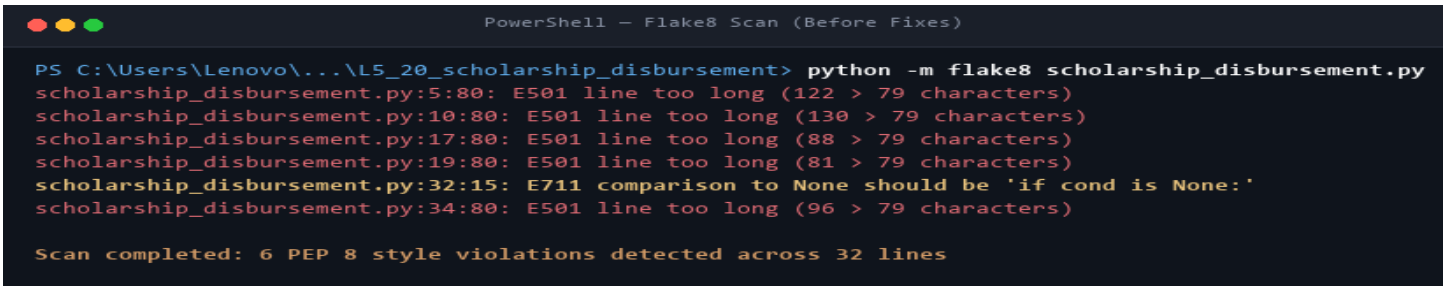


Figure 3: Pylint Initial Scan (Rated at 5.00 / 10 — Logic defects, mutable defaults, and global mutations)



3. Issue Identification & Remediation Table (from issues.md)

This table documents the technical defects detected across Bandit, Flake8, and Pylint, prioritized by severity with their corresponding remediation strategy.

Issue	Tool	Severity & Type	Line	Technical Description	Remediation / Fix Approach
SQL Injection Vector (B608)	Bandit	**Medium** Security (CWE-89)	10	Unsafe string formatting (%s, %d) dynamically constructed SQL INSERT, enabling SQL injection.	Replaced with parameterized SQL query using ? placeholders passed as parameter tuple.
Insecure Process (B605/B607)	Bandit	**Low** Security (CWE-78)	26	os.system('echo ...') spawned external system shell with partial path, vulnerable to environment tampering.	Replaced shell invocation with native Python print() and removed unused os import.
Empty Except Clause (B110 / W0718)	Bandit / Pylint	**Low** CWE-703 / Quality	27-28	try...except Exception: pass blindly swallowed all runtime exceptions, hiding system failures.	Removed redundant error suppression; standard operations execute cleanly without silencing failures.
Mutable Default Argument (W0102)	Pylint	**Medium** Logic / Bug	9	audit=[] creates a shared mutable list evaluated once at definition time, leaking state across function calls.	Set default to audit=None and dynamically initialized audit = [] inside function scope.
Singleton Comparison (E711 / C0121)	Flake8 / Pylint	Style (PEP 8)	32	Equality operator == used for None check (scheme == None) rather than identity operator.	Refactored to Python identity check: if scheme is None:.
Line Length Violations (E501 / C0301)	Flake8 / Pylint	Style (PEP 8)	5, 10, 17, 19, 34	Multiple statements exceeded the 79-character limit (up to 130 chars/line), reducing readability.	Cleanly split multi-line SQL strings, parameters, and query invocations across multiple lines under 79 chars.
Global State Mutation (W0603)	Pylint	Design / Maintainability	6, 16	Module-level variable disbursed_total was mutated using global statement in disburse().	Encapsulated database and ledger state into an object-oriented ScholarshipLedger class.
Inconsistent Returns (R1710)	Pylint	Quality / Refactoring	15-21	disburse() returned row[0] when found but implicitly fell through to None.	Added explicit return None branch for unambiguous return contract.
Naming & Docstrings (C0103/C0114)	Pylint	Convention / Quality	1, 9, 15, 31	Missing module/function docstrings and camelCase function name bySchemeCount.	Renamed to by_scheme_count, kept alias for compatibility, and added docstrings for all components.

4. Before-and-After Tool Output Comparison

Below is the consolidated comparison of the static analysis results across all three tools, capturing the before and after state from the six text reports generated in the lab.

Tool	Report File (Before Fixes)	Summary of Findings (Before)	Report File (After Fixes)	Summary of Results (After)
Bandit	reports/before/bandit_report.txt	**4 issues flagged:** • B608 (Medium): SQL injection via string formatting • B605/B607 (Low): Insecure shell call os.system • B110 (Low): Empty except Exception: pass	reports/after/bandit_report.txt	**Clean (0 issues)** All SQL statements parameterized with ? placeholders; shell calls removed.
Flake8	reports/before/flake8_report.txt	**6 violations flagged:** • 5x E501: Lines exceeding 79 characters (lines 5, 10, 17, 19, 34) • 1x E711: Comparison to None using ==	reports/after/flake8_report.txt	**Clean (0 violations)** Zero bytes output. Full PEP 8 compliance achieved across all lines.
Pylint	reports/before/pylint_report.txt	**Score: 5.00 / 10** • W0102: Dangerous mutable default [] • W0603: Global statement usage • C0114/C0116: Missing module & function docstrings • C0103: camelCase naming bySchemeCount	reports/after/pylint_report.txt	**Score: 10.00 / 10 (+5.00)** Perfect code quality score. Modular design, explicit returns, and complete docstrings.

5. Results After Fixing the Issues (Final Verification)

Figure 4: Bandit Final Scan (No issues identified — 100% clean)

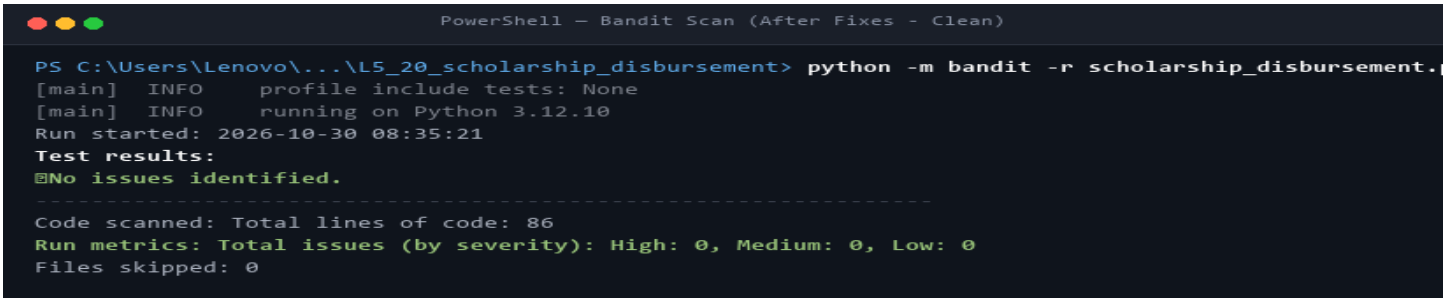


Figure 5: Flake8 Final Scan (Clean run — 0 PEP 8 violations)

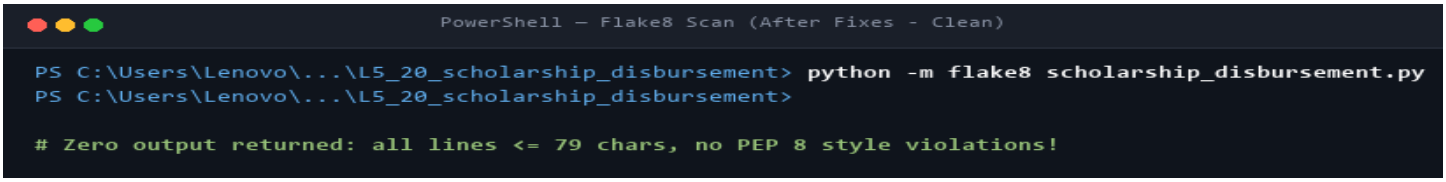


Figure 6: Pylint Final Scan (Perfect Score: 10.00 / 10 achieved)

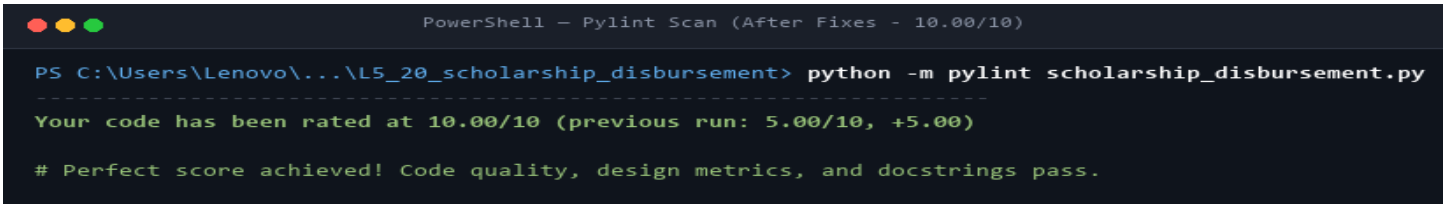
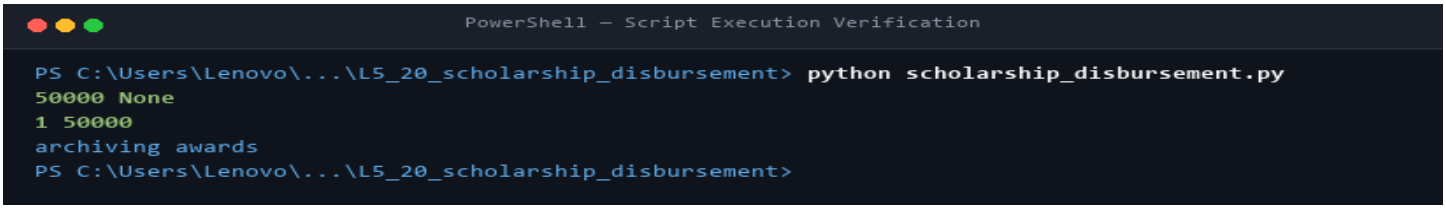


Figure 7: Functional Execution Verification (Program output matches baseline exactly)



6. Answers to Lab Manual Reflection Questions

Q1: What is the purpose and scope of static code analysis, and how does it differ from dynamic testing?

Answer: Static code analysis inspects source code at rest without executing it. It parses code into token streams and Abstract Syntax Trees (ASTs) to verify adherence to language grammar, safety standards, and architectural best practices. In contrast, dynamic testing (such as unit, integration, and end-to-end testing) executes compiled or interpreted binaries with concrete test inputs to evaluate runtime behavior. The primary advantage of static analysis is that it provides *100% path coverage* across all code paths—including untested error handlers and edge cases that dynamic tests frequently miss—and identifies critical vulnerabilities (such as SQL injection vectors or mutable defaults) early in the development lifecycle ('Shift Left'), drastically reducing remediation effort and operational risk.

Q2: How do Pylint, Bandit, and Flake8 complement each other in a modern development environment?

Answer: Each tool targets a distinct, complementary layer of software quality:
• **Bandit** acts as an *application security guard*, scanning specifically for known vulnerabilities (such as Common Weakness Enumerations / CWEs), unsafe library calls (e.g., os.system), and injection vectors.
• **Pylint** acts as a *strict code reviewer*, inspecting logical correctness, variable scoping, mutable defaults, modularity, and object-oriented design metrics.
• **Flake8** acts as a *grammar and style checker*, rapidly enforcing PEP 8 conventions, whitespace, line length limits (≤79 characters), and cyclomatic complexity.
Relying on any single tool leaves substantial blind spots. Combining all three provides comprehensive defense-in-depth: Bandit secures the application, Pylint optimizes logic and maintainability, and Flake8 guarantees readability.

Q3: Which identified issues were prioritized first and why?

Answer: Issues were strictly prioritized according to **severity and operational impact**:
1. **Medium/High Severity Security Vulnerabilities (Bandit B608):** The SQL injection vector was remediated first because raw string formatting in database queries allows malicious attackers to manipulate or exfiltrate sensitive scholarship data. In production, this represents a critical compromise.
2. **Latent Logical Bugs (Pylint W0102):** The mutable default argument audit=[] was prioritized next because in Python, default argument expressions are evaluated once when the function is defined, not on each call. Successive invocations of grant() share the exact same list instance, causing subtle state leakage between unrelated student records.
3. **System Robustness & Safety (Bandit B605/B110):** Unsafe shell invocations and silent exception swallowing were addressed to avoid environment hijacking and prevent hidden runtime crashes.
4. **Style and Formatting (Flake8 E501/E711):** Line length wrapping and comparison idioms were addressed last. While vital for readability and team collaboration, styling issues do not cause functional crashes or security exploits.

Q4: How does integrating static code analysis into CI/CD pipelines enhance overall software maintainability?

Answer: Integrating static analysis tools into automated CI/CD pipelines (e.g., GitHub Actions, GitLab CI) establishes an automated quality gatekeeper. Pull requests failing security scans (Bandit) or scoring below a predefined threshold (e.g., Pylint < 9.5) are automatically blocked from merging into the main branch. This reduces cognitive burden on human code reviewers, eliminates trivial formatting debates, prevents technical debt accumulation, and ensures that all deployed code meets stringent security and quality standards.